

PINTHAC

(Physics Informed Nuclear Thermal Hydraulics Code)

(might be a mid name but def not as bad as RETRAN REactor TRANsient...)

Introduction:

The purpose of this document is to provide a collection of ideas and later more developed plans for a thermal hydraulic analysis code with GPU and ML acceleration for rapid multi-physics coupling.

Basis:

The collective idea of this code is to incorporate GPU acceleration libraries with standard 1D analysis, as well as implement neural network approaches for rapid speedup of evaluations and optimization.

Misc Ideas and Thoughts:

- Implementation of IAPWS properties in jax and PyTorch
- Implementation of large quantities of TH correlation within JAX and PyTorch
 - High dimension PINN/DeepOnet to allow rapid single channel analysis
 - Enforce continuity and phase averaged equation
- **Inverse Problems:** Apply PINNs inversely to learn correlations for parameters
 - Symbolic regression implementation?
 - Learn void fraction and other parameters inversely
- **“SurgCorr” module** – Develop NN that approximate implicit correlations such as Chexal, Chen, Bjorge, etc. This will provide instantaneous evaluation of the correlation without numerical solving, and yields a fully differentiable model which is useful for optimization. For rigor, the surrogate can be used to produce the initial guess for traditional iteration, which would dramatically lower the required iterations for convergence
- Develop some way to allow rapid coupling to OpenMC (maybe OpenFOAM?)
- Develop an optimizer module that algorithmically optimizes the entire thermal cycle and neutronics with consideration to economic and engineering factors.
 - Develop price estimate for each operation
 - Set constraints based on standard engineering limits
 - Utilize Lagrangian methods to optimize the fully differentiable system
- Develop built in symbolic regression and operator learning module?:
 - Obtain a symbolic representation of input data for developing correlations or approximate models from simulation data
 - User can input a set of equations using sympy and under a specified syntax, obtain a NN model that learned that operator
- **Question:** Can I use a NN to approximate the IAPWS-95 iterative formulas to a higher degree of accuracy than the IAPWS-97 explicit formulation? (Press X to doubt) XXXXX
- **Other Fluids:** In the future it will be of use to implement properties and correlations for various gasses and liquids, but the code should be established for standard LWRs first
- **Efficiency of the code:** Raw efficiency will not be the primary goal for the first version of the code, nor will elegance. The codebase will just use function-level programming for data safety and simplicity, fuck object oriented programming thats for someone else (future me?) to implement once I get the initial version made
- **Backend:** Due to the large about of existing frameworks I have in PyTorch, the initial version of the code will be based on that. After the first version is developed, I

want to port the code to JAX for better efficiency, which could be done fairly easily by hand or probably almost instantly by a LLM

- Will it be better to develop each module independently and then couple?

List of Considerations and Details

I already have the IAPWS-97 implemented in PyTorch. Thus, it would be fairly trivial to convert my THCorrs library into continuous, differentiable PyTorch compatible functions. The biggest difficulty with this is dealing with correlations that require a numerical iteration. A newton solver can be written simply using autograd, but higher order techniques may be required (Chen, Bjorge were tricky with just Newton). Getting this basis of correlations implemented is important. The code modules that will be the most important to get established, in rough order of when it should be done first, are listed below:

- **Water properties IAPWS Functions**
 - Rough draft completed in PyTorch
 - Should be able to implement a monte carlo error routine which varies the outputs of the property functions within the tolerance of the IAPWS formulation error (still need to add this)
- **Other important properties/models?**
 - Thermal conductivity, diffusivity, etc of important materials such as zirc, UO2
 - Routines for iterative solvers, conductivity integrals, etc
- **Correlations module**
 - Module full of correlation functions, similar to prior THCORRs module I made, but using actual IAPWS functions instead of saturated water table
 - **Ranges and Error Bounds:** It will be very useful to go through all of the original publications where the correlations were developed and make a database of the property ranges and error of each model.
 - Have each correlation function throw a flag if an input range is violated.
 - Have a toggleable Monte Carlo routine which perturbs the outputs of the correlations by its error range. Thus the range of error can be estimated in a simulation such as SCA
- **1D FD/FVM Single Channel Analysis Module**
 - Will take some type of input file in which users specify the input parameters as well as the correlations that should be used among various steps
 - If the correlation range and error database is implemented, develop an “auto-corr” algorithm which selects the best correlation for the working range instead of the user needing to manually select one
 - Capability for time dependent code
 - Capability for differing levels of approximation
 - It should be simply to run large quantities of input cases and models/correlations
 - Rapid analysis of params over different inputs with different correlations for rapid comparison
- **PINN-TH Analysis**
 - Develop a module to allow for rapid training of a PINN for SCA
 - Make a library of various important physical equations (for example N-S, continuity, mixture equations) for quick implementation in loss functions
 - Consider starting a routine for inverse problems for discovering parameters and correlations based on TH database
- **Numerical Solver**
 - Develop a subroutine for standard numerical PDE solutions, such as the heat equation and other important PDEs
 - This can be implemented in transient SCA to model heat transfer across cladding and such

- Also implement surrogate DeepoNets/PINNs to more quickly approximate solutions
- **Neural Operator Routines**
 - Develop a module that can develop neural operators for TH problems, principally SCA
 - Have subroutines such that can generate training data from PINNs or traditional solvers and train the neural operators
- **Component Models**
 - Implement modules to model various components such as pipes, PZR, SGs, centrifugal & jet pumps, dryers, separators, reheaters, turbines, etc
- **Higher Level Modules**
 - Develop framework coupling these lower-level modules for full system analysis, like multiple interacting
 - Develop some framework for coupling the 1D representations to CFD modeling, ie 1D SCA with CFD at the inlet and exit plena for more accurate modeling (coined “1D – CFD hybrid models” in various literature sources)

Action Items

- Study and take notes on the following materials from T&K vol 2:
 - Ch. 1 – Formulation of Reactor TH Design Problem
 - Ch. 2 – Scaling of 2 Phase Flows in Reactor Systems
 - Ch. 12 – Fundamentals of Transient Reactor Sim
 - Ch. 13 – Treatment of Uncertainties in Reactor Analysis
- Read the manuals for RELAP5 and RETRAN3D
 - Understand 1D system code structure
 - Take notes on the correlations & models they implement. Consider the documentation methods/formatting they use in the manuals for me to follow
- Compose preliminary report of what models and correlations are needed ie thermal conductivities, CHF, friction, etc
- Literature review of various correlations
 - Document a brief summary, the formulation, the errors, and reference
- Compose a report documenting all correlations which will be implemented in the first draft of the code, as well as the procedure for implementing them. Also document the style of syntax the codes use.
- Produce the correlation library for the first draft of the code. I will probably implement several manually, and then give an LLM the several implementations as well as the report I wrote in order to accelerate this bit (followed by rigorous checks and combing through the code to make sure all is good and formatting is consistent with mine)
- Consider configuration syntax for the FD-SCA module. I will probably develop some domain-specific language for configurable simulations, along with an “easy” SCA mode where you can just give it an excel file like in the SCA project for a quick estimate to avoid needing to write a whole script.

- PINN module: first answer whether a pure PyTorch backend should be used or a module like DeepXDE. PyTorch is more configurable but DeepXDE can rapidly implement complex processes like resampling and such that is a pain to implement manually – See how well an LLM can implement these techniques in raw pytorch to avoid the extra labor? Write both a PyTorch module and DeepXDE module that you can switch around in?
- Develop a loss function library full of different PDEs, BCs, and other constraints
- Develop syntax for defining computational domains, sample points, correlations, and loss functions, and optimizer
- Develop plotting module to plot results